



VERDAD, BELLEZA, PROBIDAD

Tarea 1

Codificación de programas básicos

Análisis y Diseño de Algoritmos

Maestría en Ciencias e Ingeniería de Datos

Alumno: Rodrigo Piña Enríquez

Profesor: Dr. José Lázaro Martínez Rodríguez



Septiembre 2026

Índice general

1. Invertir y transponer una matriz	3
1.1. Problema	3
1.2. Solución	3
1.3. Pruebas de ejecución	4
2. Ordenamiento burbuja	5
2.1. Problema	5
2.2. Solución	5
2.3. Pruebas de ejecución	5
3. Convertir números romanos a enteros	6
3.1. Problema	6
3.2. Solución	6
3.3. Pruebas de ejecución	7
4. Máximo común divisor	8
4.1. Problema	8
4.2. Solución	8
4.3. Pruebas de ejecución	8
5. Comprobación de número primo	9
5.1. Problema	9
5.2. Solución	9
5.3. Pruebas de ejecución	9
6. Comprobación de palíndromo	10

6.1. Problema	10
6.2. Solución	10
6.3. Pruebas de ejecución	10
7. Elemento mayor de una matriz	11
7.1. Problema	11
7.2. Solución	11
7.3. Pruebas de ejecución	11
Repositorio del proyecto	12

1. Invertir y transponer una matriz

1.1. Problema

Invertir una matriz y hallar la transpuesta de una matriz.

1.2. Solución

Se implementaron dos funciones en Rust. La función `flip` intercambia los elementos simétricos de la matriz para invertir su orden. La función `transpose` intercambia filas por columnas y genera una nueva matriz transpuesta.

```
1 pub fn transpose(values: &Vec<Vec<i32>>) -> Vec<Vec<i32>> {
2     let rows = values.len();
3     let columns = values[0].len();
4     let mut transposed = vec![vec![0; rows]; columns];
5
6     for i in 0..rows {
7         for j in 0..columns {
8             transposed[j][i] = values[i][j];
9         }
10    }
11    transposed
12 }
13
14 pub fn flip(values: &mut Vec<Vec<i32>>) -> &Vec<Vec<i32>> {
15     let rows = values.len();
16     let columns = values[0].len();
17     let total = rows * columns;
18
19     for k in 0..total / 2 {
20         let mirror = total - 1 - k;
21         let i = k / columns;
22         let j = k % columns;
23         let row = mirror / columns;
24         let column = mirror % columns;
25         let temp = values[i][j];
26
27         values[i][j] = values[row][column];
28         values[row][column] = temp;
29     }
30     values
31 }
```

Código 1.1: Funciones para invertir y transponer una matriz

1.3. Pruebas de ejecución

Para ambas pruebas se utilizó la matriz de 2×3 con los valores [1, 2, 3] y [4, 5, 6].

```
$ cargo run
      MENU DE ALGORITMOS
1. Voltear matriz
2. Transponer matriz
...
Selecciona una opcion: 1
Numero de filas: 2
Numero de columnas: 3
Ingresa la fila 1: 1 2 3
Ingresa la fila 2: 4 5 6
Matriz original:
[1, 2, 3]
[4, 5, 6]
Matriz volteada:
[6, 5, 4]
[3, 2, 1]
```

Figura 1.1: Ejecución de la opción para invertir la matriz.

```
$ cargo run
      MENU DE ALGORITMOS
1. Voltear matriz
2. Transponer matriz
...
Selecciona una opcion: 2
Numero de filas: 2
Numero de columnas: 3
Ingresa la fila 1: 1 2 3
Ingresa la fila 2: 4 5 6
Matriz original:
[1, 2, 3]
[4, 5, 6]
Matriz transpuesta:
[1, 4]
[2, 5]
[3, 6]
```

Figura 1.2: Ejecución de la opción para obtener la matriz transpuesta.

2. Ordenamiento burbuja

2.1. Problema

Ordenación de la burbuja.

2.2. Solución

Se implementó el método de burbuja para ordenar un vector de números enteros de menor a mayor.

```
1 pub fn sort(values: &mut Vec<i32>) -> &Vec<i32> {  
2     for i in 0..values.len() - 1 {  
3         for j in 0..values.len() - 1 - i {  
4             if values[j] > values[j + 1] {  
5                 let temp = values[j];  
6                 values[j] = values[j + 1];  
7                 values[j + 1] = temp;  
8             }  
9         }  
10    }  
11    values  
12 }
```

Código 2.2: Función de ordenamiento burbuja

2.3. Pruebas de ejecución

Se ordenó el vector [5, 3, 8, 1, 2].

```
$ cargo run  
Selecciona una opcion: 3  
Ingresa los numeros separados por espacios:  
5 3 8 1 2  
Vector original: [5, 3, 8, 1, 2]  
Vector ordenado: [1, 2, 3, 5, 8]
```

Figura 2.3: Ejecución del ordenamiento burbuja.

3. Convertir números romanos a enteros

3.1. Problema

Convertir romanos en números enteros.

3.2. Solución

Se implementó una función que obtiene el valor de cada símbolo romano y los suma o resta de acuerdo con el símbolo siguiente.

```
1 pub fn parse(roman_value: &str) -> i32 {
2     let chars: Vec<char> = roman_value.chars().collect();
3     let mut number = 0;
4
5     for i in 0..chars.len() - 1 {
6         let now = number_equivalent(chars[i]);
7         let next = number_equivalent(chars[i + 1]);
8
9         if now < next {
10             number -= now;
11         } else {
12             number += now;
13         }
14     }
15     number += number_equivalent(chars[chars.len() - 1]);
16
17     number
18 }
19
20 fn number_equivalent(letter: char) -> i32 {
21     match letter {
22         'I' => 1,
23         'V' => 5,
24         'X' => 10,
25         'L' => 50,
26         'C' => 100,
27         'D' => 500,
28         'M' => 1000,
29         _ => 0,
30     }
31 }
```

Código 3.3: Funciones para convertir un número romano a entero

3.3. Pruebas de ejecución

Se utilizó el número romano MCMXCIV, cuyo equivalente es 1994.

```
$ cargo run
Selecciona una opcion: 4
Ingresa un numero romano: MCMXCIV
MCMXCIV = 1994
```

Figura 3.4: Ejecución de la conversión de número romano a entero.

4. Máximo común divisor

4.1. Problema

Hallar el MCD (máximo común divisor) de dos números.

4.2. Solución

Se implementó el algoritmo de Euclides para calcular el máximo común divisor de dos números enteros.

```
1 pub fn mcd(mut num1: i32, mut num2: i32) -> i32 {  
2     while num2 != 0 {  
3         let residue = num1 % num2;  
4         num1 = num2;  
5         num2 = residue;  
6     }  
7     num1  
8 }
```

Código 4.4: Función para calcular el máximo común divisor

4.3. Pruebas de ejecución

Se calcularon los divisores comunes de 48 y 18.

```
$ cargo run  
Selecciona una opcion: 5  
Ingresa el primer numero: 48  
Ingresa el segundo numero: 18  
MCD de 48 y 18 = 6
```

Figura 4.5: Ejecución del cálculo del máximo común divisor.

5. Comprobación de número primo

5.1. Problema

Dado un número n , comprobar si es primo o no.

5.2. Solución

Se implementó una función que verifica si el número tiene algún divisor entre 2 y su raíz cuadrada.

```
1 pub fn is_prime(number: i32) -> bool {  
2     if number < 2 {  
3         return false;  
4     }  
5  
6     for i in 2..=(number as f64).sqrt() as i32 {  
7         if number % i == 0 {  
8             return false;  
9         }  
10    }  
11    true  
12 }
```

Código 5.5: Función para comprobar si un número es primo

5.3. Pruebas de ejecución

Se comprobó que el número 29 es primo.

```
$ cargo run  
Selecciona una opcion: 6  
Ingresa un numero: 29  
29 es primo? true
```

Figura 5.6: Ejecución de la comprobación de número primo.

6. Comprobación de palíndromo

6.1. Problema

Comprobar si una cadena dada es un palíndromo.

6.2. Solución

Se implementó una función que compara los caracteres de la cadena desde los extremos hacia el centro.

```
1 pub fn is_palindrome(text: &str) -> bool {  
2     let chars: Vec<char> = text.chars().collect();  
3  
4     for i in 0..chars.len() / 2 {  
5         if chars[i] != chars[chars.len() - 1 - i] {  
6             return false;  
7         }  
8     }  
9     true  
10 }
```

Código 6.6: Función para comprobar si una cadena es palíndromo

6.3. Pruebas de ejecución

Se comprobó que la palabra `reconocer` es un palíndromo.

```
$ cargo run  
Selecciona una opcion: 7  
Ingresa un texto: reconocer  
reconocer es palindromo? true
```

Figura 6.7: Ejecución de la comprobación de palíndromo.

7. Elemento mayor de una matriz

7.1. Problema

Hallar el elemento mayor de una matriz.

7.2. Solución

Se implementó una función que recorre todos los elementos de la matriz y conserva el valor mayor encontrado.

```
1 pub fn mayor(values: &Vec<Vec<i32>>) -> i32 {  
2     let mut max = values[0][0];  
3  
4     for i in 0..values.len() {  
5         for j in 0..values[i].len() {  
6             if values[i][j] > max {  
7                 max = values[i][j];  
8             }  
9         }  
10    }  
11  
12    max  
13 }
```

Código 7.7: Función para obtener el elemento mayor de una matriz

7.3. Pruebas de ejecución

Se utilizó una matriz de 2×3 cuyo elemento mayor es 12.

```
$ cargo run  
Selecciona una opcion: 8  
Numero de filas: 2  
Numero de columnas: 3  
Ingresa la fila 1: 1 12 6  
Ingresa la fila 2: -4 3 8  
El elemento mayor es: 12
```

Figura 7.8: Ejecución para obtener el elemento mayor de una matriz.

Repositorio del proyecto

El código fuente, el reporte en $\text{\LaTeX}$ y los archivos correspondientes a la Tarea 1 se encuentran disponibles en el siguiente repositorio de GitHub:

https://github.com/falconmutant/algoritmos_t1